```
an_order.name = "Dave Thomas"
# ...
an_order.save
puts "The ID of this order is #{an_order.id}"
```

The `new` constructor creates a new `Order` object in memory; we have to remember to save it to the database at some point. Active Record has a convenience method, `create`, that both instantiates the model object and stores it into the database:

rails80/e1/ar/new_examples.rb

```
an_order = Order.create(
  name:     "Dave Thomas",
  email:    "dave@example.com",
  address:  "123 Main St",
  pay_type: "check")
```

You can pass `create` an array of attribute hashes; it'll create multiple rows in the database and return an array of the corresponding model objects:

rails80/e1/ar/new_examples.rb

```
orders = Order.create(
  [ { name:     "Dave Thomas",
      email:    "dave@example.com",
      address:  "123 Main St",
      pay_type: "check"
    },
    { name:     "Andy Hunt",
      email:    "andy@example.com",
      address:  "456 Gentle Drive",
      pay_type: "po"
    } ] )
```

The *real* reason that `new` and `create` take a hash of values is that you can construct model objects directly from form parameters:

```
@order = Order.new(order_params)
```